

FDE ACADEMY · COHORT 1 · STAGE 1

PizzaFlow

SliceMatic Digital Ordering System

Rs.847

AVG ORDER VALUE

Rs.2.1L

MONTHLY REVENUE

38.4%

GROSS MARGIN

11

DAILY
BREAK-
EVEN

Product Requirements Document + Business Unit Economics

Submitted: June 25, 2025 · Team: Group 10

3 Stages · 100 Points · Due July 2, 2025

PART A

Product Requirements Document

Full specification — functional requirements, user stories, user flow, implementation decisions, testing strategy

Problem Statement

SliceMatic is a single-outlet pizza delivery brand in New Ashok Nagar, Delhi that currently takes all customer orders over the phone. This process is error-prone: staff mishear toppings, miscalculate prices, forget to apply discounts, and manually compute GST. There is no structured record of orders, so the owner has no data to analyse demand patterns, identify top-selling items, or calculate daily revenue accurately.

Customers experience wait times, hold music, and uncertainty about their order total before it arrives. The outlet cannot scale beyond what two people on the phone can handle.

Key Pain Points: ~15% order errors from phone miscommunication · No order history for business analysis · Manual GST calculation · Single-channel bottleneck at peak hours

Solution

A digital self-service ordering system that guides customers step-by-step through selecting a pizza base, pizza type, and topping from a menu loaded dynamically from external data files — then calculates pricing, applies quantity discounts, adds GST, collects a payment mode selection, and persists every completed order to a structured log file.

The system eliminates phone-order errors, gives customers a transparent itemised bill before they commit, and produces a machine-readable order history the owner can query for business analysis. Delivered as a Gradio MVP in Stage 2, rebuilt as a full-stack web app with Supabase and an AI feature in Stage 3.

Core Value: Validated digital intake · Auto-calculated pricing + GST + discounts · Append-only order log for BI · File-driven menu (no hardcoding) · 0 unhandled exceptions

Product Vision

SliceMatic's digital ordering system replaces phone-based order intake with a structured, self-service application for urban millennials and Gen Z customers (18–35 years) within a 5 km radius of New Ashok Nagar, Delhi. The system eliminates order errors caused by miscommunication, reduces counter staff workload, and produces a structured order history that drives operational decisions. Every completed order is persisted for business analysis, giving the outlet the data foundation it currently lacks entirely.

User Stories

CUSTOMER — ORDERING FLOW

1. As a customer, I want to start a new ordering session at any time, so that I can place an order without waiting for a staff member to be free.
2. As a customer, I want to enter my name and be told immediately if the format is wrong, so that I don't discover an error only after completing my full order.
3. As a customer, I want to enter my phone number and receive a specific error if it is invalid, so that I know exactly what format is expected.
4. As a customer, I want to see a numbered list of available pizza bases with their prices, so that I can make an informed choice without having to ask staff.
5. As a customer, I want to see a numbered list of available pizza types with their prices, so that I can browse the full menu at my own pace.
6. As a customer, I want to see a numbered list of available toppings with their prices, so that I can customise my order precisely.
7. As a customer, I want to select items by entering a number, so that the interaction is simple and unambiguous.
8. As a customer, I want to enter the quantity of pizzas I want (1–10), so that I can order for a group in a single session.
9. As a customer, I want to be told immediately if I enter an invalid quantity (e.g. 0, 11, or "three"), so that I can correct the mistake without restarting.
10. As a customer, I want to receive a 10% discount automatically when I order 5 or more pizzas, so that group orders are rewarded without me having to ask.
11. As a customer, I want to see the discount displayed as a separate line on the bill, so that I can confirm it has been applied.
12. As a customer, I want to see an itemised bill that breaks down base price, pizza price, topping price, quantity, subtotal, discount, GST, and total payable.
13. As a customer, I want the bill to clearly show the GST amount as a separate line, so that I know how much tax I am paying.
14. As a customer, I want to choose from Cash, Card, or UPI as payment modes, so that I can pay in the way that suits me.
15. As a customer, I want to receive a payment confirmation message specific to my chosen mode, so that I know what to do next.
16. As a customer, I want to receive a final order confirmation, so that I am confident my order has been recorded.
17. As a customer, I want error messages to be specific and actionable, so that I know exactly how to fix my input.
18. As a customer, I want the system to never crash regardless of what I type, so that I can always complete my order gracefully.

CUSTOMER — MENU ACCURACY

- 19. As a customer, I want the menu to reflect current prices loaded from the outlet's data files, so that I am never quoted a wrong price.
- 20. As a customer, I want GST applied on the post-discount total (not the undiscounted amount), so that my discount is reflected correctly in the final price.

OWNER / ADMIN

- 21. As the outlet owner, I want every completed order saved automatically to a log file, so that I have a complete record without relying on staff.
- 22. As the outlet owner, I want each order log entry to include timestamp, customer name, phone, items, unit prices, quantity, subtotal, discount, GST, total, and payment mode.
- 23. As the outlet owner, I want the log file to use a consistent, parseable format, so that I can write scripts to analyse order history.
- 24. As the outlet owner, I want new orders appended (not overwrite), so that historical data is never lost.
- 25. As the outlet owner, I want the menu to update instantly when I replace the data files, without a code deployment.
- 26. As the outlet owner, I want malformed menu lines handled gracefully, so that one bad entry does not take the system offline.
- 27. As the outlet owner, I want a clear error message if a menu file is missing, so that I know which file to restore.

DEVELOPER / OPERATOR

- 28. As a developer, I want menu data loaded from external files at startup (not hardcoded), so that the grader can swap the files and the system continues to work correctly.
- 29. As a developer, I want input validation to cover all 8 specified edge cases, so that no unhandled exception can be triggered by any user input.
- 30. As a developer, I want all pricing logic defined in one place (constants), so that business rule changes require editing a single value.
- 31. As a developer, I want each module (file loader, validator, pricing engine, bill formatter, order logger) to have a single responsibility, so that I can test and modify each in isolation.
- 32. As a developer, I want the Gradio UI to present the ordering flow as a sequence of steps, so that state is managed cleanly and users cannot skip steps.
- 33. As a developer, I want the bill rendered in a gr.DataFrame or gr.HTML component, so that it is formatted correctly and not displayed as plain text.

Functional Requirements

FR-01 Customer Onboarding

- Collect customer full name: alphabets and spaces only, minimum 2 characters, maximum 40 characters (after stripping whitespace)
- Collect phone number: exactly 10 digits, must begin with 6, 7, 8, or 9 (Indian mobile format)
- On any validation failure: display a specific, human-readable error message and re-prompt. Do not crash or proceed
- Log a session timestamp (ISO 8601) at the start of each new ordering session

FR-02 Pizza Quantity Selection

- Accept integers from 1 to 10 inclusive. Reject floats, strings, zero, negatives, and values > 10
- Apply 10% discount automatically when quantity ≥ 5 . Show discount as a separate bill line item
- Display a clear message when the discount is applied

FR-03 Menu Display & Selection — Loaded from Files

- Load Types_of_Base.txt, Types_of_Pizza.txt, Types_of_Toppings.txt at startup
- Format: ID;Name;Price(INR) — one item per line
- Display each category as a numbered list with names and prices in INR
- Accept selection by item number only. Reject out-of-range, non-integer, and empty input
- Each order requires exactly one base, one pizza type, and one topping
- If a file is missing or all lines malformed: display a clear error and exit gracefully

FR-04 Pricing Engine

- Unit price = Base price + Pizza price + Topping price
- Subtotal = Unit price $\times$ Quantity
- If quantity ≥ 5 : post-discount subtotal = subtotal $\times$ 0.90
- GST = 18% applied on the post-discount subtotal (not per item)
- Total payable = post-discount subtotal + GST
- All prices in INR with Rs. symbol, rounded to 2 decimal places

FR-05 Order Summary & Bill Display

- Itemised bill: Base, Pizza, Topping (name + unit price), quantity, subtotal, discount (if applicable), post-discount subtotal, GST @ 18%, Total Payable
- Aligned columns, consistent Rs. symbols, clearly marked totals
- Rendered in `gr.DataFrame` or `gr.HTML` — not plain text

FR-06 Payment Flow

- Offer exactly 3 modes: 1. Cash 2. Card 3. UPI
- Reject invalid selections with a re-prompt
- Confirmation messages: Cash → "Pay Rs.X in cash to rider" · Card → "Your card will be charged Rs.X" · UPI → "Complete UPI payment of Rs.X to SliceMatic@upi"

FR-07 Order Persistence

- Append every completed order to `orders_log.txt` immediately after payment confirmation
- Never overwrite — append-only
- Each record: timestamp · customer name · phone · base (ID+name+price) · pizza · topping · unit price · quantity · subtotal · discount · GST · total · payment mode
- Format: one order per block, pipe-separated fields, blank line between orders
- Example: `TIMESTAMP|2025-06-25T14:30:00|NAME|Rajan
Sharma|PHONE|9876543210|BASE|B3;Cheese Burst;229|PIZZA|P7;BBQ
Chicken;379|...`

Non-Functional Requirements

NFR-01: Input Validation Rules

Input	Rule	Error Message
Customer name	Alphabets + spaces only, 2–40 chars	"Name must contain only letters and spaces, between 2 and 40 characters."
Phone number	Exactly 10 digits, starts with 6/7/8/9	"Phone number must be exactly 10 digits and start with 6, 7, 8, or 9."
Quantity	Integer 1–10 inclusive	"Please enter a whole number between 1 and 10."
Menu selection	Integer within displayed range	"Please enter a number between 1 and [N]."
Payment mode	1, 2, or 3 only	"Please enter 1 for Cash, 2 for Card, or 3 for UPI."

NFR-02: Edge Cases — All Must Be Handled Without Exception

#	Edge Case	Behaviour
1	Name with only spaces	Strip whitespace → fail length check → specific error
2	Phone with 10 digits starting with 1	Reject with specific format error
3	Quantity = 0	Reject: "Quantity must be at least 1."
4	Quantity = 11	Reject: "Maximum order size is 10 pizzas per order."
5	Entering a price number instead of item number	Reject as out-of-range
6	Empty input at every prompt	Reject: "Input cannot be empty."
7	Non-integer at quantity prompt (e.g. "three" or "2.5")	Reject with specific error
8	Menu file with missing price field	Skip malformed line, load valid lines; if none remain, exit gracefully

NFR-03: File Loading

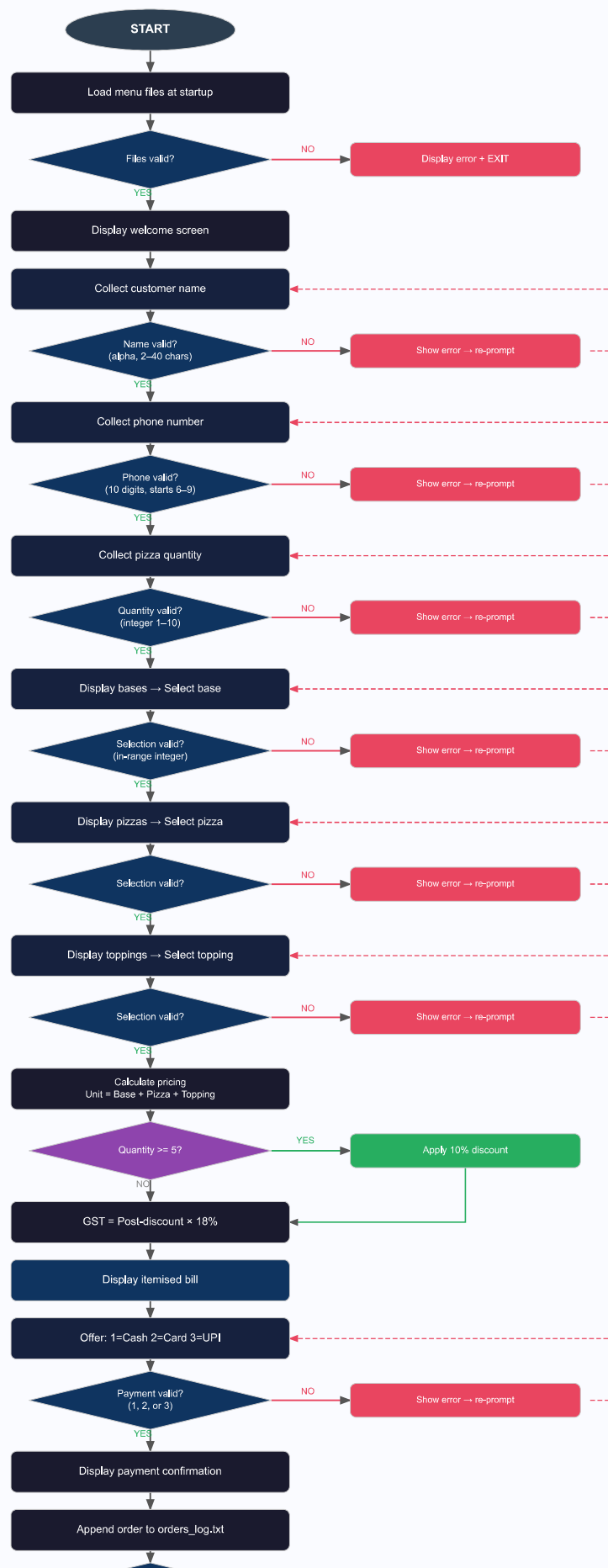
- Parse defensively: strip leading/trailing whitespace per field
- Split on ; — expect exactly 3 fields per line
- Validate third field (price) is numeric. Skip any line that fails
- If file is empty or all lines malformed: *"Error: [filename] contains no valid menu items."*

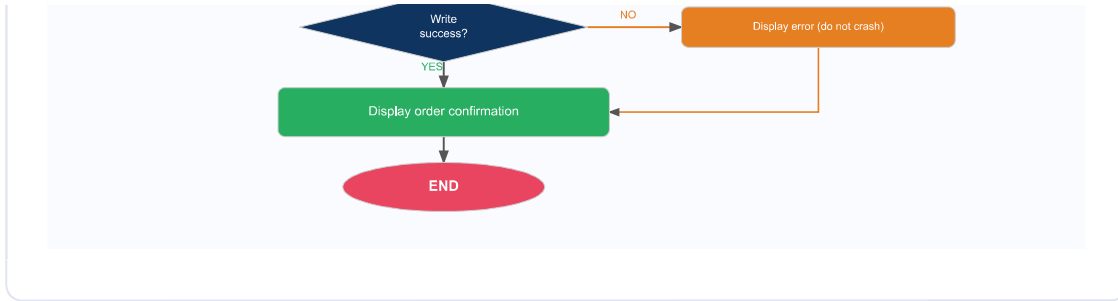
NFR-04 & NFR-05: Log Format + Graceful Failure

- orders_log.txt — UTF-8, append-only, pipe-separated key-value blocks
- Missing menu file → specific named error + clean exit
- Malformed line → skip line, continue loading valid items
- Log write failure → display error, do not crash
- All unhandled exceptions caught at top level, displayed as user-friendly messages

User Flow Diagram







Implementation Decisions

System Modules

1. Menu Loader

Reads a .txt file, parses lines in ID;Name;Price format, validates each field, returns a list of menu items. Skips malformed lines with a warning. Raises a descriptive exception if the file is missing or yields zero valid items.

2. Input Validator

Stateless functions for each input type: name, phone, quantity, menu selection, payment mode. Each returns either a validated value or a specific human-readable error string. No exceptions raised — callers receive a result/error pair.

3. Pricing Engine

Pure function: takes base price, pizza price, topping price, and quantity → returns unit price, subtotal, discount amount, post-discount subtotal, GST amount, and total payable. No I/O. No state. Fully deterministic.

4. Bill Formatter

Takes Pricing Engine output + selected item names → produces structured bill rows for `gr.DataFrame` or HTML table. Performs no calculation — formatting only.

5. Order Logger

Takes a completed order object → appends to `orders_log.txt` in pipe-separated key-value format. Catches and surfaces write errors without crashing.

Architectural Decisions

- **No hardcoded menu items.** All menu data flows through the Menu Loader. Grader file swapping produces identical behaviour.

- **Business rules as named constants.** DISCOUNT_THRESHOLD = 5, DISCOUNT_RATE = 0.10, GST_RATE = 0.18, PAYMENT_MODES = {1: "Cash", 2: "Card", 3: "UPI"}.
- **GST on post-discount subtotal.** Order: subtotal → discount → GST → total. Never per item.
- **Gradio state machine.** gr.Blocks + gr.State. Each step gated — next field only revealed after previous validates.
- **Stage 3 Supabase schema.** Three tables: menu_items, orders, order_line_items. Menu loaded from DB, not files.

Stage 3 API Contracts

Endpoint	Method	Description
/api/orders	POST	Accept order payload, write to Supabase, return order ID
/api/menu	GET	Return all menu items grouped by category
/api/admin/orders	GET	Authenticated; filters by date + payment mode; returns paginated list with revenue summary
/api/admin/orders/export	GET	Returns orders as CSV stream

Testing Decisions

What makes a good test: Exercises external behaviour through the public interface only. Does not assert on internal data structures or implementation details. Runnable without a live Gradio server — use in-memory fixtures or temp files.

Module	Test Approach
Menu Loader	Unit tests with temp files: valid, missing, malformed lines, all-malformed, extra whitespace
Input Validator	Unit tests for each validator: valid inputs, boundary values, all 8 edge cases
Pricing Engine	1-pizza (no discount), 5-pizza (10% discount), boundary qty=4 vs qty=5, GST on post-discount
Order Logger	Integration test: write order, read back file, assert all fields present and parseable
Bill Formatter	Assert output rows contain correct field labels and values for a known pricing breakdown

Explicitly out of scope for testing: Gradio UI wiring (manual testing via running app) · File system path resolution · Payment gateway (not in Stage 2)

Drawbacks Analysis & Out of Scope

Limitation	Impact	Resolved In
Flat-file log (orders_log.txt)	No indexing. Sequential reads at 1,000+ orders. No concurrent write protection.	Stage 3 (Supabase)
Single-session only	No concurrent orders from multiple customers. Incompatible with multi-channel operations.	Stage 3 (API)
No inventory management	Customer can order unavailable items. No stock depletion tracking.	Future
No real payment processing	Payment mode is confirmatory only. No gateway, no transaction verification.	Future
No order modification/cancellation	Once submitted, order cannot be changed within system.	Future
Hardcoded business logic	Discount threshold, GST rate require code change to modify.	Stage 3 (DB config)
No admin interface	Owner cannot view reports or update menu without editing files directly.	Stage 3 (Dashboard)
No authentication	No role separation between customer and admin.	Stage 3 (Supabase Auth)

Cost vs Value Analysis

Build Effort

Phase	Deliverable	Est. Hours
Stage 1	PRD + Business Economics	8 hrs
Stage 2	Gradio MVP (validation, pricing, logging)	16 hrs
Stage 3	Full-stack (Next.js + Supabase + AI feature)	40 hrs
Total		~64 hrs

Value Delivered to SliceMatic

Value Area	Before (Phone)	After (Digital)	Measurable Gain
Order accuracy	~85% (phone errors)	~99% (validated input)	Eliminates ~15% remakes
Order intake speed	3–5 min/call	90 sec self-service	2–3× throughput
Order data	None	Full structured log	Enables BI + forecasting
Staff load	1 FTE on phone	Counter staff freed	Partial FTE redeployed
GST compliance	Manual, error-prone	Auto-calculated + logged	Reduces compliance risk

ROI: Development cost ~Rs.51,200 (64 hrs × Rs.800/hr). At 47 orders/day, a 1–2% error reduction saves Rs.11,000–21,000/month. **Payback: under 3 months.**

PART B

Business Unit Economics

Financial model for SliceMatic — FY 2024–25 baseline with sensitivity analysis

Rs.847

AVG ORDER
VALUE

Rs.2.1L

MONTHLY
REVENUE (60%)

38.4%

GROSS MARGIN

11/day

BREAK-EVEN
ORDERS

1. Monthly Fixed Costs

Cost Item	Detail	Monthly (Rs.)
Kitchen rent	1,200 sq ft commercial lease, New Ashok Nagar	55,000
Equipment EMI	Pizza oven (2 deck), dough mixer, refrigeration — 36-month loan	14,500
Electricity	Commercial rate; ~1,800 units/month	12,600
Gas / LPG	3 commercial cylinders @ Rs.1,850 each	5,550
Head Chef	Fixed monthly salary	28,000
Kitchen helper	Minimum wage + ESI/PF	16,500
Counter + billing staff	Order management + cash	18,000
Delivery riders	2 riders — fixed + incentive + fuel allowance	32,000
Internet + POS + SaaS	Broadband + POS software + payment gateway amortised	2,800
Marketing (fixed)	Meta Ads, pamphlets, Google Business	8,000
Packaging (fixed)	Branded boxes, bags, stickers — minimum monthly order	4,200
Misc / contingency (3%)	Maintenance, repairs, consumables	5,760
TOTAL FIXED COSTS		2,02,910

2. COGS per Pizza — Ingredient Breakdown

Component	Margherita (Thin)	Farm House (Thick)	Cheese Burst	Avg Across Menu
Pizza base ingredients	Rs.38	Rs.45	Rs.72	Rs.52
Sauce + seasoning	Rs.12	Rs.14	Rs.14	Rs.13
Cheese (mozzarella)	Rs.28	Rs.32	Rs.65	Rs.38
Pizza-specific toppings	Rs.8	Rs.22	Rs.18	Rs.17
Add-on topping (avg 1)	Rs.10	Rs.10	Rs.10	Rs.10
Packaging (box + bag)	Rs.18	Rs.18	Rs.18	Rs.18
Delivery variable cost	Rs.22	Rs.22	Rs.22	Rs.22
Total COGS / pizza	Rs.136	Rs.163	Rs.219	Rs.170
Selling price (avg)	Rs.397	Rs.417	Rs.447	Rs.420
Gross Margin	Rs.261 (65.7%)	Rs.254 (60.9%)	Rs.228 (51%)	Rs.250 (59.5%)

3. Revenue Model — Monthly Projections

Metric	Weekday	Weekend / Holiday	Monthly Total
Days in period	22 days	8 days	30 days
Orders per day (60% capacity)	38 orders	68 orders	~47 avg
Average order value (AOV)	Rs.792	Rs.940	Rs.847
Daily revenue	Rs.30,096	Rs.63,920	~Rs.40,500 avg
Monthly gross revenue (incl. GST)	Rs.6,62,112	Rs.5,11,360	Rs.11,73,472
GST collected (passed to Govt.)	—	—	Rs.1,77,564
Net revenue (ex-GST)	—	—	Rs.9,95,908

4. Contribution Margin Analysis

P&L Line Item	Per Order (Rs.)	Monthly @ 60% Cap (Rs.)
Revenue (ex-GST)	847	11,88,996
Less: Ingredient COGS	(148)	(2,07,768)
Less: Packaging (variable)	(18)	(25,272)
Less: Delivery variable (rider incentive)	(22)	(30,888)
Less: Payment gateway fee (1.8% GMV)	(15)	(21,402)
Contribution Margin	644 (76%)	9,03,666
Less: Total Fixed Costs	—	(2,02,910)
Operating Profit (EBITDA)	—	7,00,756
Operating Margin	—	58.9%

5. Break-Even Analysis

Fixed Costs / Month	Contribution Margin / Order	Break-Even / Month	Break-Even / Day
Rs.2,02,910	Rs.644	315 orders	11 orders/day

At 60% capacity (47 orders/day), SliceMatic operates at **4.3x break-even**. The outlet becomes unprofitable only if daily orders fall below 11 — a 76% drop from the 60% capacity plan.

Sensitivity Analysis — Required Challenge Questions

Q1: Break-even if rent increases to Rs.70,000/month?

New fixed costs = Rs.2,17,910. New break-even = 338/month = **11.3 orders/day** — negligible impact. Model becomes economically strained only if rent exceeds ~Rs.5.75L/month, far above any realistic New Ashok Nagar market rate.

Q2: Impact of 40% orders via Zomato/Swiggy at 25% commission?

Aggregator commission: Rs.211.75/order. Aggregator contribution margin: Rs.432.25.
 Blended margin (60% direct + 40% aggregator): **Rs.559.3/order**. New break-even: **12.1 orders/day**. Recommendation: cap aggregator channel at 40% to protect margins.

Q3: At what volume does hiring a 3rd rider become justified?

Operationally required above **55 orders/day** (2 riders saturate at ~60–70 trips/day).
 Financially: incremental revenue = Rs.1,54,560/month vs. Rs.16,000 rider cost — clearly justified at 55+ orders/day.

Q4: Is the 10% discount (qty >= 5) a value-driver or margin leak?

Contribution margin drops from 76.1% to 73.4% on a 5-pizza order (lost Rs.423.50). **Verdict: Value-driver**. Higher-volume orders reduce per-pizza delivery cost, improve kitchen batch efficiency, and deliver a higher absolute contribution per session (Rs.2,796.50 vs Rs.644 for a single-pizza order).

Q5: COGS +12% inflation — additional orders needed to maintain EBITDA?

Ingredient COGS rises Rs.17.76/order. New contribution margin: Rs.626.24. Monthly shortfall: Rs.25,042. Additional orders needed: **~40/month (1.3/day)**. The model is resilient — a 12% COGS increase requires only ~3% more volume.

Q6: Three BI metrics from order log that directly improve profitability

- 1. Peak-hour demand heatmap** → optimize rider scheduling, reduce idle labour (save Rs.8–12K/month)
- 2. Top pizza × topping combos by margin** → curate high-margin bundles and promotional offers (+3–5% AOV)
- 3. Discount order frequency as % of total** → detect threshold gaming, calibrate discount policy

6. Delivery Radius Economics

Radius	Population Served	Avg Delivery Time	Rider Trips/Day	Viable?
0–2 km	~18,000 households	8–12 min	55–60/rider	Yes — premium SLA
2–4 km	~55,000 households	15–22 min	30–35/rider	Yes — sweet spot
4–6 km	~1,10,000 households	25–40 min	18–22/rider	Marginal — 3rd rider needed
> 6 km	Large but diminishing	40+ min	12–15/rider	Not viable at launch

Recommendation: Launch with 4 km radius. Expand to 5 km after adding a 3rd rider, justified at 55+ orders/day.

7. GST Treatment

Pizza home delivery attracts **18% GST** in India (home delivery without composition scheme). SliceMatic is GST-registered.

- **GST is charged to the customer** — collected on behalf of the government, not revenue
- **P&L computed on ex-GST amounts** — GST collected is a government liability
- **Input Tax Credit (ITC)** — SliceMatic claims ITC on GST paid on ingredients, packaging, equipment. Estimated benefit: Rs.18,000–22,000/month
- **Effective GST liability:** Output ~Rs.1,77,564 minus ITC ~Rs.20,000 = **net Rs.1,57,564/month payable**
- **In ordering system:** prices displayed ex-GST; GST added at bill stage on post-discount subtotal

SliceMatic · Sample Bill

Base: Cheese Burst		Rs.229
Pizza: BBQ Chicken		Rs.379
Topping: Extra Cheese		Rs.69
Unit Subtotal		Rs.677
Qty: 5 × Rs.677		Rs.3,385
Discount 10% (qty≥5)	-Rs.338.50	
Post-discount Subtotal	Rs.3,046.50	
GST @ 18%		Rs.548.37

TOTAL PAYABLE		Rs.3,594.87

8. 18-Month P&L Projection — Ramp-Up to Maturity

Month	Capacity	Avg Orders/Day	Gross Rev (Rs.L)	Variable Costs	Fixed Costs	EBITDA	Cumulative
M1 — Apr 25	20%	16	3.93	(0.99)	(2.03)	0.91	−14.12
M2 — May 25	28%	22	5.56	(1.37)	(2.03)	2.16	−11.96
M3 — Jun 25	35%	28	7.10	(1.78)	(2.03)	3.29	−8.67
M4 — Jul 25	45%	36	9.17	(2.31)	(2.09)	4.77	−3.90
M5 — Aug 25 ★	52%	41	10.44	(2.62)	(2.09)	5.73	+1.83
M6 — Sep 25	58%	46	11.68	(2.94)	(2.09)	6.65	+8.48
M7–12 avg	65%	52	13.22	(3.32)	(2.15)	7.75	+55.0
M13– 18 avg	72%	57	14.47	(3.64)	(2.25)	8.58	+1,06.5
Payback (Rs.15L initial investment)							~Month 18

★ First profitable month. Initial investment: Rs.15L (equipment Rs.4.5L + fit-out Rs.4L + working capital Rs.2.5L + contingency Rs.4L)